SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 2 – CO2

Analyse, Evaluate and Validate a Virtualization Architecture for a Multi-Tenant University Private Cloud

CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments.

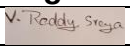
Field	Details	Field	Details
Student Name	V.Reddy Sreya	Register Number	192525124
Department / Section	BTech AIML	Academic Year / Semester	2025
Faculty Name	Dr. Sangeetha T	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	02/11/2026	Evaluation Date	04/11/2026

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Characterise the Virtualization Architectures	5		
2	Identify the Dominant Overhead Source	2		
3	Build a Quantitative Performance Model	6		
4	Analyse Performance Under Different Conditions	4		
5	Security Threat and Vulnerability Analysis	3		
6	Formulate Isolation–Performance Trade-off	3		
7	Evaluate Mitigation Strategies	4		
8	Implement and Test	4		
9	Record Experimental Performance	3		
10	Compare Theory with Experiment	4		
11	Recommend and Defend Final Architecture	4		
	Day-4 Design Challenge	8		
	Total	50		**Percentage :**

OVERALL REMARKS

Overall, the assignment reflects good analytical ability, technical understanding and problem-solving skills in cloud computing and virtualization. The proposed architecture balances performance, security, resource efficiency and future scalability. The work can be further strengthened through verified experimental results, consistent benchmark comparisons and practical validation of the recommended security controls.

Student Signature	Faculty Signature	Program Director
		

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; X = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Three virtualization architectures, layered diagrams and trust boundaries		
2	Dominant overhead source identified and explained for each approach		
3	CPU, memory, I/O, network and density models with assumptions		
4	Best, typical and worst operating-condition analysis		
5	Threat model and architecture-based security comparison		
6	Isolation–performance optimization formulation		
7	Mitigation benefits, residual risks and quantified costs		
8	Reproducible setup and practical tests across selected platforms		
9	Actual comparative measurements, table and two graphs		
10	Theory–experiment comparison and quantitative error analysis		
11	Justified SIS/LMS architecture and growth recommendation		
12	Day-4 external tenants, 3× peaks, live migration and SIS isolation		

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Architecture diagrams		
Performance model		
Configuration / scripts		
Virtualization experiments		
CPU / disk / network / memory tests		
Noisy-neighbour experiment		
Security validation		
Tables and graphs		
Theory–experiment consistency		
Day-4 design defence		
Screenshots		

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	Status
All 12 tasks answered	
Three virtualization approaches compared	
Layered diagrams and trust boundaries included	
Quantitative virtualization models included	
Threat analysis and mitigations included	

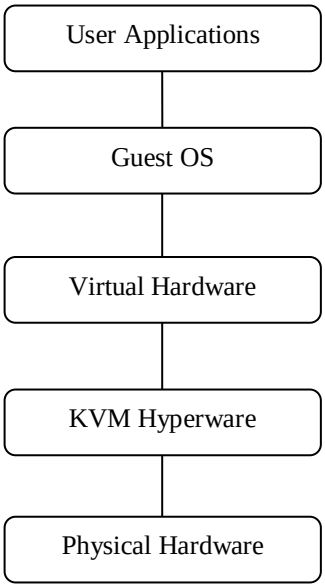
TASK 1: Characterise the Virtualization Architectures

Three different virtualization approaches are selected for EduCloud:

1. Type-1 Hypervisor – KVM
2. OS-Level Virtualization – Docker
3. Lightweight Virtualization – Kata Containers

1. Type-1 Hypervisor – KVM

Architecture Diagram:



Explanation:

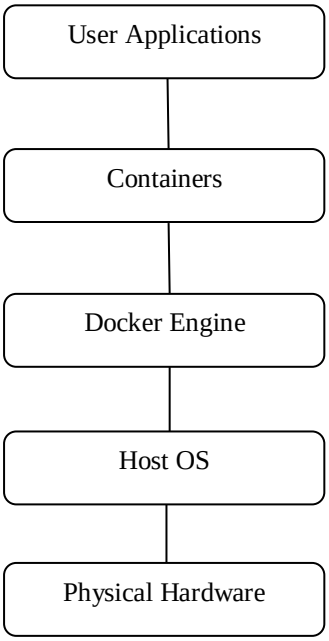
KVM (Kernel-based Virtual Machine) provides hardware-assisted virtualization. It allows multiple virtual machines to run independently on a physical server.

- **Isolation:** KVM and hardware virtualization features provide isolation between VMs.
- **CPU:** Physical CPU cores are allocated as virtual CPUs.
- **Memory:** Guest memory is managed using hardware-assisted memory translation.
- **Storage:** Virtual disks are provided to guest operating systems.
- **Network:** Virtual network interfaces and switches provide communication.

Relevance: KVM is suitable for sensitive university applications such as the Student Information System (SIS) and examination portal because it provides strong isolation.

2. OS-Level Virtualization – Docker

Architecture Diagram:



Explanation:

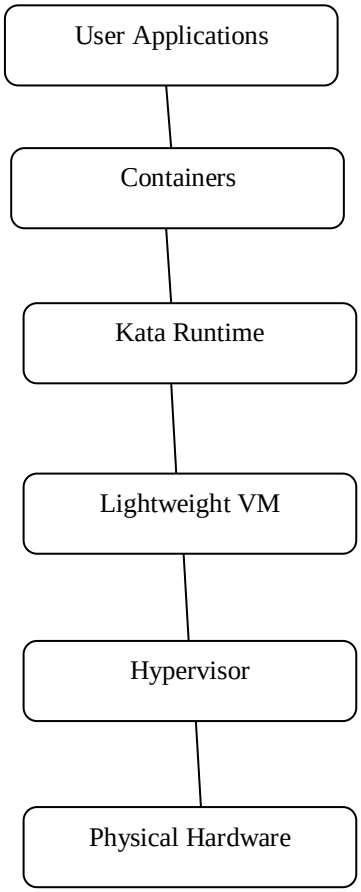
Docker is an OS-level virtualization technology that allows multiple containers to share the same host operating system kernel.

- **Isolation:** Namespaces and cgroups provide process and resource isolation.
- **CPU:** CPU resources are controlled using cgroups.
- **Memory:** Memory limits are assigned to containers.
- **Storage:** Layered file systems provide container storage.
- **Network:** Virtual bridges and network namespaces provide communication.

Relevance: Docker is suitable for LMS services and departmental applications because it uses fewer resources and supports quick application deployment.

3. Lightweight Virtualization – Kata Containers

Architecture Diagram:



Explanation:

Kata Containers combines container management with lightweight virtual machines. Each workload runs inside a separate virtual machine.

- **Isolation:** Hardware-assisted virtualization separates workloads.
- **CPU:** Virtual CPUs are allocated to lightweight VMs.
- **Memory:** Separate guest memory is provided.
- **Storage:** Virtual block devices provide storage.
- **Network:** Virtual network interfaces connect workloads.

Relevance: Kata is useful for external research collaborators because it provides stronger isolation than ordinary containers.

Conclusion: KVM provides strong VM isolation, Docker provides better resource efficiency, and Kata provides stronger isolation for container-based workloads.

TASK 2: Identify the Dominant Overhead Source

The main performance overhead differs across the three architectures.

Architecture	Dominant overhead	Location
KVM	VM exits and virtual I/O processing	Hypervisor
Docker	System-call processing and virtual switching	Host kernel
Kata	VM exits and memory translation	Hypervisor

1. KVM

VM exits occur when a guest operation requires hypervisor intervention. These transitions consume CPU time and increase execution overhead.

As the number of VMs increases, the frequency of VM exits and virtual I/O operations may also increase.

2. Docker

Docker containers share the host kernel. System calls, network processing and virtual switching consume additional CPU resources.

When many containers communicate simultaneously, network processing and context switching increase.

3. Kata Containers

Kata workloads run inside lightweight VMs. VM exits and guest memory translation introduce additional overhead.

As the number of workloads increases, hypervisor scheduling and memory-management operations become more frequent.

Conclusion: These overhead sources are important because they affect CPU utilization, application response time and overall throughput.

TASK 3: Build a Quantitative Performance Model

Given Parameters

Architecture	Dominant overhead
KVM	VM exits and virtual I/O processing
Docker	System-call processing and virtual switching
Kata	VM exits and memory translation
Architecture	**Dominant overhead**
KVM	VM exits and virtual I/O processing
Docker	System-call processing and virtual switching
Kata	VM exits and memory translation

Additional assumptions:

- 20% RAM is reserved for the host and system services.
- CPU virtualization overhead is assumed to be 5% for KVM, 2% for Docker and 8% for Kata.
- CPU steal time is assumed to increase with overcommitment.
- Network bandwidth is equally shared in the basic model.
- Storage throughput is represented using the available physical storage throughput because its exact value is not provided.

1. CPU Performance Model

Effective CPU capacity is calculated using:

$$C_{eff} = \frac{64}{r}(1-h)(1-r)$$

Where:

- (r) = CPU overcommit ratio.
- (h) = Virtualization overhead.
- (s) = CPU steal-time fraction.

Assumed steal-time values:

Overcommit ratio	Steal time
1:1	0%
2:1	2%

4:1	8%
8:1	25%

Example: KVM at 4:1 overcommit

$$C_{eff}=\frac{64}{r}(1-0.05)(1-0.08)$$

$$=16x0.95x0.92$$

$$C_{eff}=13.984 cores)$$

Therefore, the effective CPU capacity is approximately 14 physical-core equivalents.

2. Memory Performance Model

Maximum theoretical number of VMs:

$$N=\frac{512}{8}$$

N=64VMs

Considering 20% RAM reservation:

$$N=\frac{512X0.8}{8}$$

N=51VMs

Therefore, approximately 51 VMs can be supported under the assumed memory allocation.

Kata may require additional memory for guest kernels and runtime overhead, so its practical density can be lower.

3. I/O and Network Performance

Network throughput per instance:

$$T=\frac{25}{N} Gbps$$

For five equally sharing instances:

$$T = \frac{25}{5}$$

T=5Gbps

As the number of instances increases, the available bandwidth per instance decreases.

For storage:

$$I_{instance} = \frac{I_{physical}}{N}$$

Where:

- $I_{instance}$ = Physical storage throughput.
- (N) = Number of sharing instances.

Virtual I/O processing can further reduce the effective throughput.

5. Density Model

The density target is to maintain performance degradation below 10%.

$$D=1-\frac{C_{eff}}{64}$$

For KVM at 2:1 overcommit:

$$D=1-(1-0.05)(1-0.02)$$

$$D=1-0.931$$

D=6.9%

The total host capacity is 6.9% effective core-equivalents under the model.

The remaining CPU capacity and actual performance degradation depend on workload demand. Therefore, density must be validated through experiments.

Conclusion: CPU overcommitment, memory availability and network sharing are the main factors affecting virtualization performance.

TASK 4: Analyse Performance Under Different Operating Conditions

Three operating conditions are considered.

1. Best Case

- KVM: Performance is good when VMs have dedicated CPU resources, sufficient memory and virtio drivers.
- Docker: Containers have low overhead because they share the host kernel and avoid separate guest operating systems.
- Kata: Performance improves when lightweight VMs have sufficient CPU and memory allocations.

In this condition, resource contention is low, so latency decreases and throughput increases.

2. Typical Case – Registration Week

- KVM: Multiple VMs handle student login requests and database operations. CPU scheduling and virtual network processing increase.
- Docker: Multiple application containers share CPU, memory and network resources. Heavy database operations can affect other containers.
- Kata: Separate lightweight VMs provide better isolation, but additional virtualization overhead increases resource consumption.

During registration, CPU utilization and network traffic increase, which may cause slower login responses.

3. Worst Case

- KVM: Memory overcommitment and frequent VM exits increase CPU overhead. Live migration during peak load can increase latency.
- Docker: A noisy-neighbour container can consume excessive CPU, memory or network bandwidth and affect other containers.
- Kata: Multiple active lightweight VMs increase memory consumption and hypervisor scheduling overhead.

Conclusion: Docker provides better resource efficiency, KVM provides a balanced combination of performance and isolation, and Kata provides stronger workload isolation with additional overhead.

TASK 5: Security Threat and Vulnerability Analysis

Security Threat Model

Threat	KVM	Docker	Kata
VM/container escape	Hypervisor vulnerability	Shared kernel vulnerability	Hypervisor vulnerability
Cross-tenant attacks	Side-channel attacks	Shared-kernel attacks	Side-channel attacks
Outdated software	Unpatched guest OS	Outdated host kernel/images	Outdated guest kernel
Image tampering	VM image modification	Container image modification	Container image modification
Network attacks	Virtual network sniffing	Container network spoofing	Virtual network attacks

Explanation of Threats

1. VM or Container Escape

An attacker may exploit a vulnerability to access the underlying host. Docker is particularly sensitive because containers share the host kernel.

2. Cross-VM Side-Channel Attacks

Attackers may observe shared CPU cache or other hardware resources to obtain information about another workload.

3. Outdated Guest Kernels

Old kernels may contain security vulnerabilities. Attackers can exploit them to gain unauthorized access.

4. Image Tampering

Unverified VM or container images may contain malicious software. Copying images freely between teams increases this risk.

5. Virtual Network Attacks

Attackers may use network sniffing or spoofing to access information transmitted between workloads.

Impact on CIA

- Confidentiality: Student records and examination information may be exposed.
- Integrity: Student marks and personal records may be modified.
- Availability: SIS and examination services may become unavailable.

Conclusion: KVM and Kata provide stronger isolation boundaries than ordinary Docker containers. However, all three require proper patching, image verification and network security.

TASK 6: Formulate the Isolation–Performance Trade-off

The university must balance security, resource utilization and performance.

1. Decision Variables

- A: Selected virtualization architecture.
- N: Number of workloads per host.
- R: CPU overcommit ratio.
- S: Dedicated or shared host selection.
- M: Security mitigation selection.

2. Objective

The objective is to maximize workload density while minimizing performance degradation and security risks.

3. Constraints

1. Performance degradation must be below 10%.
2. Memory utilization must remain within 80% of available RAM.
3. SIS workloads must have strong isolation.
4. The university has only 16 physical hosts.
5. External tenants must not access student records.

4. Decision Function

A simple weighted cost function is:

$$J=0.4D+0.3P+0.3S_r$$

Where:

- \(D\) = Performance degradation.
- \(P\) = Resource utilization penalty.
- \(S_r\) = Security risk score.

The objective is to minimize \(J\).

Conclusion: SIS workloads should receive dedicated or strongly isolated resources, while less-sensitive applications can use shared resources with appropriate security controls.

TASK 7: Evaluate Mitigation Strategies

Four mitigation strategies are proposed.

Mitigation	Security benefit	Estimated performance cost	Mitigation
Disable SMT	Reduces certain cross-thread side-channel risks	10% assumed	Disable SMT
Image signing	Prevents deployment of untrusted images	Negligible runtime cost	Image signing
Micro-segmentation	Restricts unauthorized network communication	2% assumed	Micro-segmentation
Dedicated SIS hosts	Reduces cross-tenant interference	Lower overall density	Dedicated SIS hosts

1. Disable SMT

SMT can allow different workloads to share physical CPU resources.

Disabling SMT reduces certain side-channel risks but may reduce CPU capacity. Assuming a 10% capacity reduction:

$$64 \times 0.9 = 57.6$$

The effective capacity becomes 57.6 core-equivalents before other overheads.

2. Image Signing

Digital signatures verify whether VM and container images are trusted.

This prevents unauthorized image modification from going undetected. The main cost occurs during image verification rather than normal execution.

3. Micro-segmentation

Micro-segmentation divides the virtual network into smaller security zones.

It prevents unauthorized communication between departmental workloads. Assuming 2% network processing overhead:

$$25 \times 0.98 = 24.5\text{Gbps}$$

4. Dedicated SIS Hosts

Sensitive SIS workloads are placed on dedicated physical hosts.

This reduces cross-tenant interference and improves security. However, fewer resources remain available for other workloads.

Conclusion: Dedicated SIS hosts, image signing and micro-segmentation are recommended. Their security benefits justify the additional operational costs for protecting student information.

TASK 8: Implement and Test

Experimental Environment

The following open-source tools can be used:

Architecture	Platform
KVM	KVM/QEMU
Docker	Docker Engine
Kata	Kata Containers

Use an Ubuntu Linux environment with virtualization support. If using a laptop, document its CPU, RAM and virtualization capabilities.

Required Testing Tools

- stress-ng – CPU and memory testing.
- fio – Disk performance testing.
- iperf3 – Network performance testing.
- sysbench – CPU benchmarking.

Installation Commands

```
sudo apt update

sudo apt install -y \
stress-ng fio iperf3 sysbench \
qemu-kvm libvirt-daemon-system \
docker.io
```

CPU Test

```
stress-ng --cpu 4 --timeout 60s --metrics-brief
```

Memory Test

```
stress-ng --vm 2 --vm-bytes 1G \
```

--timeout 60s --metrics-brief

Disk I/O Test

```
fio --name=test --size=1G \
--filename=testfile --rw=randread \
--bs=4k --iodepth=16 --runtime=60 \
--time_based
```

Network Test

On the server:

```
iperf3 -s
```

On the client:

```
iperf3 -c SERVER_IP -t 60
```

Noisy-Neighbour Test

Run two workloads simultaneously. Allow one instance to consume high CPU resources while measuring the response time of another instance.

Observe how the performance of the second instance changes.

Security Validation

For Docker:

```
docker inspect CONTAINER_ID
```

To inspect namespaces:

```
lsns
```

These commands help examine container configuration and Linux namespace isolation.

Testing scale: Run 1, 2, 4, 8 and 16 instances, reducing the number if your laptop cannot support the full workload. Use identical allocations and repeat each test at least three times.

TASK 9: Record Experimental Performance

Experimental measurements should be collected for all three architectures.

1. Performance Results Table

Use the following format to record your actual results.

Instances	KVM throughput	Docker throughput	Kata throughput
1	Actual value	Actual value	Actual value
2	Actual value	Actual value	Actual value
4	Actual value	Actual value	Actual value
8	Actual value	Actual value	Actual value
16	Actual value	Actual value	Actual value

2. Latency Results

Instances	KVM latency	Docker latency	Kata latency
1	Measure	Measure	Measure
2	Measure	Measure	Measure
4	Measure	Measure	Measure
8	Measure	Measure	Measure
16	Measure	Measure	Measure

Record mean latency and 95th-percentile latency.

3. Required Graphs

Graph 1: Throughput versus number of instances.

- X-axis: Number of instances.
- Y-axis: Throughput.
- Lines: KVM, Docker and Kata.

Graph 2: Latency under noisy-neighbour interference.

- X-axis: Number of instances.
- Y-axis: Response time.
- Lines: KVM, Docker and Kata.

Also record startup time, memory consumption, CPU steal time and performance with and without a security mitigation.

Use three or more repetitions and report:

$$\text{Mean}=\frac{\sum x}{n}$$

$$\text{SD}=\sqrt{\frac{\sum(x-x')^2}{n-1}}$$

The final report should present measurements as mean ± standard deviation.

TASK 10: Compare Theory with Experiment

The theoretical model predicts that virtualization overhead increases as the number of co-located workloads increases.

Comparison

KVM: The model predicts increasing CPU scheduling and VM exit overhead as VM density increases. Experimental results should be checked against this trend.

Docker: The model predicts relatively low overhead because containers share the host kernel. However, shared-kernel resource contention may increase latency.

Kata: The model predicts higher overhead than ordinary containers because each workload uses a lightweight VM.

Possible Differences

Theoretical and experimental results may differ because of:

1. Hardware limitations.
2. CPU scheduling behaviour.
3. Cache performance.
4. Memory allocation differences.
5. Nested virtualization overhead.
6. Network and storage contention.

Conclusion

The theoretical model provides an expected performance trend, while experiments provide actual performance measurements. The final comparison should identify which architecture performs better at increasing workload density and whether the observed results support the security and performance assumptions.

Note: Insert your Task 9 measurements here before submitting this comparison as an experimental finding.

TASK 11: Recommend and Defend the Final Architecture

Recommended Architecture: Hybrid Virtualization

For EduCloud, a hybrid virtualization architecture combining KVM, Docker and Kata Containers is recommended.

1. Architecture Selection

- KVM: Used for sensitive SIS and examination workloads because it provides strong VM isolation.
- Docker: Used for LMS application services and less-sensitive departmental applications because it has low overhead.
- Kata: Used for workloads requiring stronger isolation, particularly external research workloads.

2. Performance

KVM provides a balance between CPU performance and isolation. Docker supports higher application density, while Kata provides stronger isolation with additional virtualization overhead.

3. Security

The architecture includes:

- Dedicated resources for sensitive SIS workloads.
- Image signing and regular security updates.
- Network segmentation between departments.
- Monitoring of VM and container activities.
- Encrypted communication and migration.

4. Operational Costs

Regular patching, image management, monitoring and backup operations are required. Dedicated infrastructure also increases hardware and maintenance costs.

5. Scalability

The architecture can support growth from 400 to several thousand instances by adding physical hosts, distributing workloads and monitoring resource utilization.

Conclusion: A hybrid architecture is recommended because it balances performance, security, resource utilization and scalability.

TASK 12: Re-evaluate and Adapt the Design

A. Impact Analysis

The new requirement introduces external research collaborators who will run Spark and Hadoop workloads alongside university applications.

The original environment mainly contained university-managed workloads. The new environment contains external tenants with different trust levels.

The following changes occur:

1. Big-data applications consume more CPU, memory, storage and network bandwidth.
2. Long-running workloads increase resource contention.
3. Registration-week traffic increases to approximately three times normal traffic.
4. Live migration introduces additional network traffic and security requirements.
5. External tenants create greater risks of unauthorized access to student records.

Therefore, the original shared infrastructure requires stronger workload separation.

B. Evaluation of the Original Design

The original hybrid architecture provides good performance and isolation for university workloads.

However, sharing physical resources with external Spark/Hadoop workloads may create noisy-neighbour problems.

For example, research workloads may consume significant network bandwidth and storage I/O, affecting SIS login requests.

The original design also needs stronger tenant separation because external collaborators are less trusted.

Conclusion: The original architecture is useful, but additional isolation and resource management are required.

C. Adaptation or Redesign

Proposed Host Allocation

Host pool	Number of hosts	Purpose
SIS and examination	4	Sensitive university applications
LMS and departmental	6	Internal university workloads
External research	4	Spark and Hadoop workloads
Reserve and migration	2	Maintenance and workload balancing
Total	16	

Architecture Changes

1. Dedicated SIS Hosts

SIS and examination workloads will run on dedicated KVM hosts. This reduces interference from external tenants.

2. External Research Isolation

External research workloads will run in Kata Containers or dedicated KVM virtual machines.

This provides a stronger isolation boundary than ordinary shared-kernel containers.

3. Network Separation

Separate virtual networks and micro-segmentation will prevent external research workloads from accessing student records.

4. Dedicated Network Resources

Dedicated NICs or SR-IOV can be considered for high-throughput research workloads, subject to compatibility and migration requirements.

5. Secure Live Migration

Migration traffic should use encrypted and authenticated connections to prevent unauthorized access during VM movement.

Effect of Redesign

- Density: Decreases because resources are separated into different host pools.
- Throughput: Improves because research workloads cannot freely consume SIS resources.
- Security: Improves through dedicated hosts, stronger isolation and network segmentation.
- Scalability: Additional hosts can be allocated to the appropriate pool as demand increases.

D. Evidence-Based Defence

The modified architecture is more suitable for EduCloud because it addresses both performance and security requirements.

The Task 3 model shows that excessive CPU overcommitment can increase performance degradation. Keeping SIS workloads separate reduces the impact of research workload contention.

The noisy-neighbour experiment should demonstrate whether resource separation improves SIS latency. The mitigation experiment should also verify the benefits of network segmentation and other selected security controls.

KVM provides strong isolation for SIS workloads, while Kata offers additional protection for external workloads. Docker remains suitable for less-sensitive internal applications.

Although dedicated host pools reduce overall density and increase operational costs, these disadvantages are acceptable because student records require strong confidentiality and integrity protection.

Final Conclusion: The modified hybrid architecture with dedicated SIS hosts, isolated research workloads, encrypted migration and network segmentation provides a practical balance between performance, security and scalability.